SMART EXPENSE TRACKER – FINAL REPORT

---

1. Title Page

Course Name: Object-Oriented Programming (Java)
Project Title: Smart Expense Tracker
Student Name: Yaman Yusuf
Roll Number: _77_____
Instructor Name: Mr_Susovan Jana_____
Semester: Spring 2026

---

2. Problem Statement (Max 1 Page)

In today's fast-paced world, managing personal finances has become increasingly complex. Many individuals fail to keep track of their income and expenses, leading to poor budgeting and financial instability. Traditional methods such as manual record-keeping are inefficient and prone to errors.

The objective of this project is to design and implement a **Smart Expense Tracker** using Java, based on Object-Oriented Programming principles. The system enables users to record, categorize, and analyze their financial transactions efficiently.

The application provides functionalities such as:

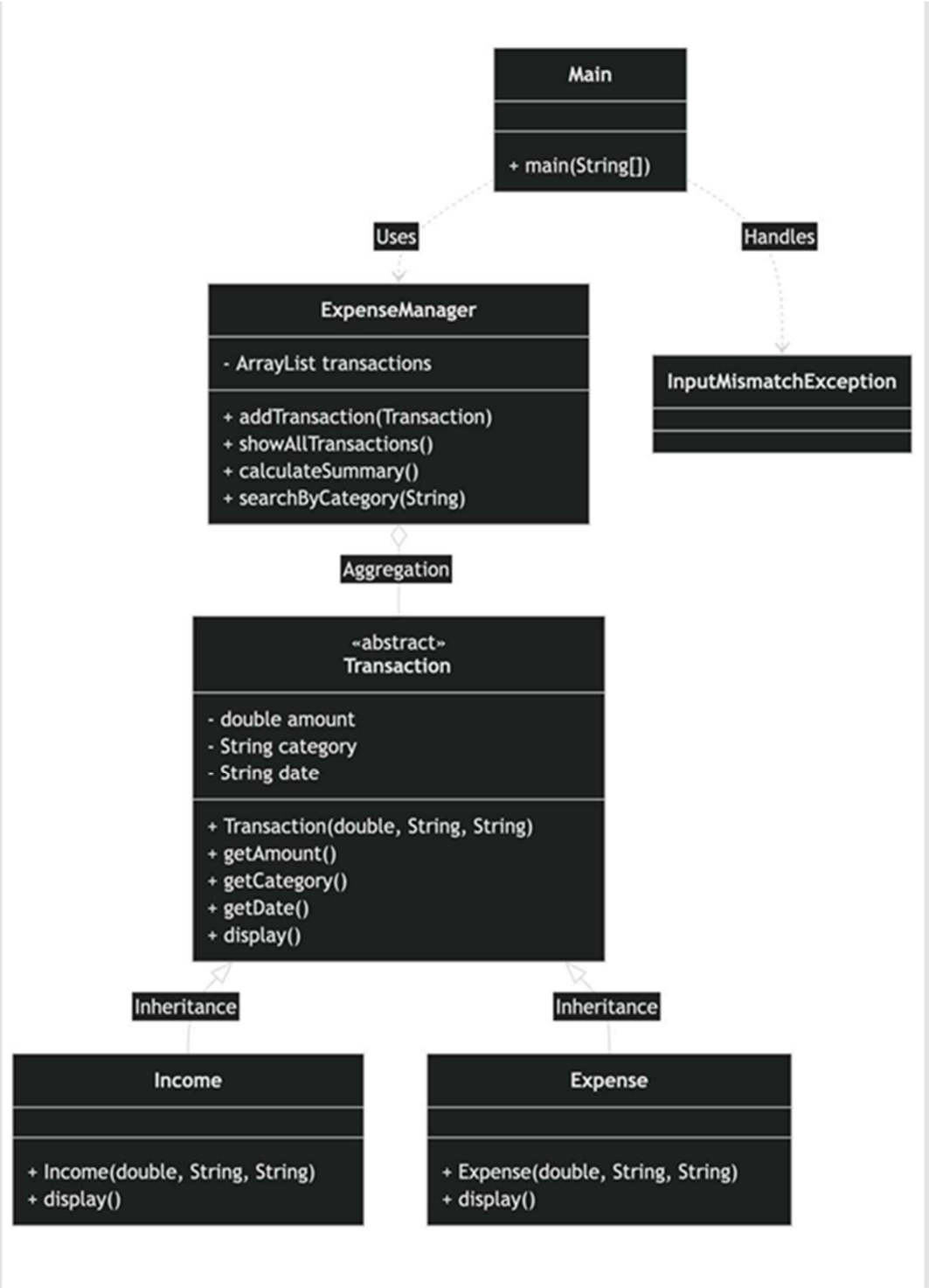
- Adding income and expense transactions
- Viewing transaction history
- Calculating total income, expenses, and balance
- Searching transactions by category

This system aims to offer a structured and scalable solution for managing personal finances.

---

3. System Design

3.1 UML Class Diagram (Text Representation)



3.2 Description of Major Classes

- **Transaction Class**
  - Abstract base class
  - Contains common attributes: amount, category, date
  - Defines abstract method display()

- **Income Class**
  - Inherits Transaction
  - Represents incoming money
  - Overrides display() to show positive amount

- **Expense Class**
  - Inherits Transaction
  - Represents outgoing money
  - Overrides display() to show negative amount

- **ExpenseManager Class**
  - Core logic handler
  - Stores all transactions
  - Performs calculations and search operations

- **Main Class**
  - Entry point of program
  - Handles user interaction
  - Controls application flow

3.3 Package Structure

project/

├── Transaction.java

├── Income.java

├── Expense.java

├── ExpenseManager.java

└── Main.java

3.4 Interaction Flow

User Input

↓

Main.java

↓

Create Object (Income/Expense)

↓

ExpenseManager

↓

Processing (store / calculate / search)

↓

Output Display

---

4. OOP Concepts Demonstrated

4.1 Abstraction

Where used: Transaction class

Why:
To define a common structure while hiding implementation details

Snippet:

```
public abstract class Transaction {
    public abstract void display();
}
```

Explanation:
The abstract method forces child classes to define their own display logic.

4.2 Inheritance

Where used: Income and Expense

Why:
To reuse code and avoid duplication

Snippet:

```
public class Expense extends Transaction {
    public Expense(double amount, String category, String date) {
        super(amount, category, date);
    }
}
```

Explanation:
Child classes inherit properties and behavior from Transaction.

4.3 Polymorphism

Where used: displayAll()

Why:
To allow same method call to behave differently

Snippet:

```
for (Transaction t : transactions) {
    t.display();
}
```

Explanation:
Runtime decides which display() to execute.

4.4 Encapsulation

Where used: Transaction

Why:
To protect data

Snippet:

```
private double amount;


public double getAmount() {
    return amount;
}
```

Explanation:
Direct access is restricted; controlled access provided.

---

5. Implementation Highlights

5.1 Use of ArrayList

ArrayList<Transaction> transactions;

👉 Allows dynamic storage of multiple transaction types

5.2 Runtime Type Checking

if (t instanceof Income)

👉 Separates income from expense during calculations

5.3 Input Validation

if (amount <= 0)

👉 Prevents invalid entries

5.4 Exception Handling

catch (InputMismatchException e)

👉 Prevents program crash

---

6. Testing & Error Handling

6.1 Test Cases

Case	Input	Output
Add Income	5000	Stored
Add Expense	2000	Stored
Summary	-	Correct result

6.2 Edge Cases

- Negative amount ❌
- Empty category ❌
- No transactions ✓ message

6.3 Failure Handling

- Invalid input handled
- Safe program execution

---

7. Git Workflow

Development Steps

1. Project initialization
2. Transaction class creation
3. Income & Expense implementation
4. ExpenseManager logic
5. Main UI development
6. Testing and bug fixes

Version Control Benefits

- Track changes
- Maintain history
- Enable collaboration

---

8. Conclusion

The Smart Expense Tracker demonstrates effective use of Object-Oriented Programming concepts. The system is modular, scalable, and easy to extend. It successfully handles financial data in a structured manner.

---

9. Future Scope

- GUI using JavaFX
- Database integration
- Graphical reports
- Mobile app version
- Authentication system

Appendix

A. **Project Proposal**

Smart Expense Tracker System – Project Report

Target Users / Audience:

- College students managing monthly allowances
- Young professionals tracking daily spending
- Individuals who want to monitor personal finances
- Anyone looking for a simple budgeting tool

Problem Statement:
Many students and young professionals struggle to manage their daily expenses and often lose track of where their money is spent. This project aims to develop a Smart Expense Tracker System using Java that allows users to record, categorize, and analyze their expenses efficiently. The system provides a simple interface where users can add income, record expenses, view spending history, and generate basic summaries. It helps users understand their financial habits and encourages better money management while demonstrating core object oriented programming concepts.

Core Features:

- Add expenses with category, amount, and date
- Record income sources such as salary or pocket money
- View complete transaction history
- Generate monthly expense summaries
- Search transactions by category or date
- Store data using Java Collections (ArrayList)
- Handle invalid inputs using exception handling

OOP Concepts Used:

- Abstraction – Abstract Transaction class
- Inheritance – Expense and Income classes extend Transaction
- Polymorphism – Overriding methods for transaction display
- Encapsulation – Private variables with getters and setters
- Exception Handling – Managing invalid user input
- Collections – ArrayList for storing transactions

B.

B. Approval Mail

PCCCS495 Proposal Review – APPROVED_MINOR

From: soumadip.biswas@ieminternational.ai soumadip.biswas@ieminternational.ai
To: MdYaman Yusuf MdYaman.Yusuf2024@iem.edu.in
Sent: Monday 9 March at 9:36 am

Dear Md Yaman Yusuf ,

Project: Smart Expense Tracker System
Decision: APPROVED_MINOR
Score: 82

Strengths:
The proposal clearly defines a practical and relatable problem (personal expense tracking) and targets a relevant audience (students, young professionals). It outlines a good set of essential features that directly address the stated problem, making the application genuinely useful. The explicit mention and planned use of core OOP concepts like Abstraction, Inheritance, Polymorphism, Encapsulation, Exception Handling, and Java Collections are excellent for a university project and demonstrate a strong understanding of the learning objectives. The proposed modular architecture, with a clear separation of concerns (Transaction hierarchy, ExpenseManager, UserInterface), indicates a sound approach to object-oriented design.

Improvements:
1. **Data Persistence:** The most critical missing element is how transaction data will be stored persistently between application runs. For an 'expense tracker,' data persistence is fundamental. Consider implementing file I/O (e.g., CSV, JSON, object serialization) or a simple embedded database (e.g., SQLite, H2) to save and load user data.
2. **User Interface Specification:** Clarify whether the 'UserInterface' will be a Command-Line Interface (CLI) or a Graphical User Interface (GUI) (e.g., using Swing or JavaFX). This decision significantly impacts the scope, design, and complexity of the UI development.
3. **Testing Strategy:** Include a plan for how the application's functionality will be tested. Mentioning unit tests for core logic (e.g., `ExpenseManager`) and potentially integration tests would strengthen the

---

C. Additional Submissions

(If applicable)